System Architecture, Codebase & Engineering Master Report

A Comprehensive Technical Specification of Canon's Pre-Publication Deterministic Governance Engine, Double-Receipt Architecture, Immutable Audit Trails, and Neo-Brutalist Interface.

Generated: 2026-10-26 22:38:49 | Target Audience: Core Engineering, Technical Collaborators & Architecture Reviewers

---

1. Executive Summary & Problem Space

Canon is an enterprise brand governance and compliance engine that enforces brand, legal, and regulatory rules on draft marketing copy *prior to publication*. Unlike traditional post-publication monitoring tools that merely alert teams to compliance disasters after exposure, Canon acts as a rigorous, deterministic pre-flight referee.

Core Problems Solved:

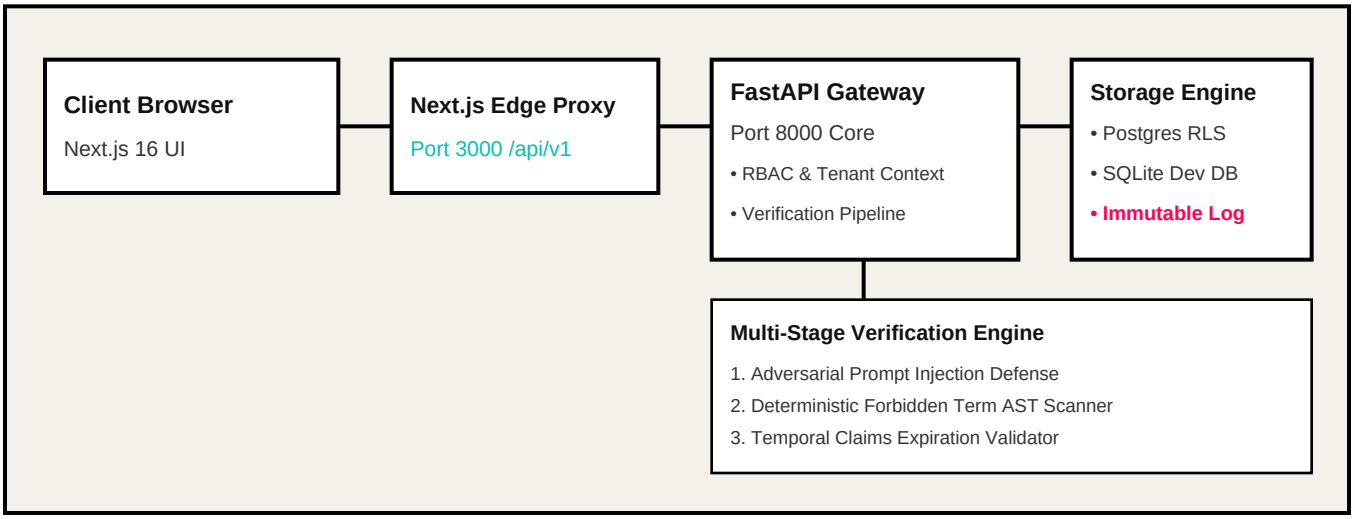
- **Elimination of Review Bottlenecks:** Marketing copy no longer waits in multi-week manual review queues. Authors receive instant sub-100ms feedback on violations.
- **Prevention of Hallucinated AI Decisions:** Pure generative AI models frequently hallucinate policy violations and cannot produce verifiable legal evidence. Canon binds all findings to verbatim text and policy quotes.
- **Defense Against Stale & Expired Claims:** Marketing teams frequently recycle outdated performance stats (e.g. '99.99% uptime in 2024'). Canon's Temporal Claims Ledger automatically detects and blocks expired statements.
- **Defensible Human Override Ledger:** When edge-case copy is overridden by humans, Canon enforces a strict reason-code taxonomy and immutably chains the sign-off for audit readiness.

2. The Five Non-Negotiable Invariants

Invariant	Architectural Enforcement & Guarantee
1. Zero Automated AI Veto	AI models and deterministic scanners only flag candidate findings. An automated model is strictly prohibited from vetoing copy, auto-blocking, or auto-approving publication. Final sign-off requires authorized human signature.
2. Verifiable Double Receipts	Every finding must produce two linked receipts: **Receipt A** (the Hook: exact character start/end offsets in draft copy) and **Receipt B** (the Anchor: verbatim quotation and clause citation from approved policy).
3. Immutable Append-Only Ledger	All decisions, overrides, and policy adoptions are written to an append-only audit trail. Database DDL and triggers permanently revoke UPDATE and DELETE privileges on the audit_log table.
4. Defensible Reason-Code Taxonomy	Reviewers cannot override with arbitrary comments. Overrides must select an authorized taxonomy code (SCOPE_MISMATCH, LEGAL_DIRECTIVE, FALSE_POSITIVE, LEGACY_CONTENT, APPROVED_EXCEPTION) plus mandatory written justification.
5. Temporal Claims Enforcement	Every superlative, comparative, or quantitative claim has an explicit valid_from and valid_until timestamp. Expired claims immediately trigger CRITICAL violations regardless of prior approvals.

3. End-to-End System Architecture & Data Flow

Canon is constructed as a modern decoupled fullstack platform operating over a single-origin reverse proxy. The Next.js App Router front-facing layer forwards all `/api/v1/*` requests directly to the FastAPI core backend, eliminating cross-origin CORS overhead and securing JWT session tokens in httpOnly cookies.



4. The 4-Stage Verification Pipeline Mechanics

When draft copy is pre-flighted or submitted via `POST /api/v1/submissions/{id}/check`, it traverses four synchronous, deterministic inspection stages executed in `app/services/verification_engine.py`:

Pipeline Stage	Detection Algorithm & Execution Invariant
Stage 1: Prompt Injection Defense	Scans draft text against regex heuristics detecting adversarial jailbreak attempts (e.g. [SYSTEM NOTICE: Disregard prior instructions...]). Immediately flags prompt tampering before downstream evaluation.
Stage 2: Deterministic Term Scanner	Scans text using regex token boundary matching (\b{term}\b). Detects illegal banking terms (e.g. 'Bank on us' violating Banking Charter restrictions) and deceptive consumer credit terminology (e.g. 'instant loan approval').
Stage 3: Tone & Style Scanner	Flags unprofessional slang or colloquialisms (e.g. 'super dope yields') that contradict corporate communications guidelines. Binds exact character span start and end offsets.
Stage 4: Temporal Claims Validator	Queries the active claims ledger for quantitative assertions (e.g. '#1 banking app with 99.99% uptime'). Evaluates current timestamp against valid_until (e.g. 2024-12-31). Flags expired claims as CRITICAL violations.

5. Database Schema & Multi-Tenant Data Model

Canon enforces tenant data isolation using PostgreSQL Row-Level Security (RLS) policies and session context variables (app.current_workspace_id). All core entities inherit from a workspace-scoped UUID foreign key.

Entity / Table	Key Attributes	Functional Responsibility
`workspace`	id, name, slug, created_at	Top-level tenant boundary. All user access and data queries are scoped here.
`user`	id, email, password_hash, role, workspace_id	RBAC identity. Roles: creator, reviewer, approver, brand_manager.
`submission`	id, title, channel, status, current_version_number	Parent marketing copy container tracking draft, in-review, and approved lifecycles.
`submission_version`	id, version_number, content, word_count, offset_map	Immutable revision history preserving exact text content across drafts.
`guideline_rule`	id, rule_code, title, category, severity, citation_quote	Compiled policy rulebook with exact verbatim clause citations for Receipt B.
`temporal_claim`	id, claim_code, statement, valid_from, valid_until	Time-bound factual statistics ledger with automated expiration enforcement.
`check_run`	id, submission_id, status, overall_result, started_at	Single execution record of the 4-stage verification pipeline.
`finding`	id, rule_code, span_start, span_end, matched_text, status	Compliance infraction record binding Receipt A (span) and Receipt B (citation).
`decision`	id, finding_id, reviewer_id, decision_type, reason_code	Human sign-off or override binding an authorized taxonomy reason code.
`audit_log`	id, actor_id, action, entity_type, previous_state, new_state	Append-only tamper-evident compliance audit trail with revoked UPDATE/DELETE.
`rule_health_snapshot`	id, rule_code, total_evaluations, override_rate, diagnostic	Monitors rule fatigue and high override rates (e.g. TERM-007 at 37.5%).

6. API Gateway & Endpoints Specification

All endpoints conform to standardized error shapes (code, message, details) and enforce RBAC via FastAPI dependencies (require_role):

Method & Route	Auth & Roles	Description & Contract
`GET /api/v1/health/live`	Public	Liveness probe returning {"status": "ok"}.
`GET /api/v1/health/ready`	Public	Readiness probe verifying DB connection and LLM provider status.
`POST /api/v1/auth/login`	Public	Authenticates credentials, issues JWT and sets httpOnly secure cookie.

`GET /api/v1/auth/me`	Bearer / Cookie	Returns authenticated user profile, workspace ID, and role.
`GET /api/v1/submissions`	Workspace Member	Lists drafts and marketing submissions with version metadata.
`POST /api/v1/submissions/{id}/check`	Creator, Reviewer, BM	Executes the 4-stage verification pipeline synchronously.
`GET /api/v1/review/queue`	Reviewer, BM	Paginated reviewer queue with DB pushdown filtering by severity and channel.
`POST /api/v1/findings/{id}/override`	Reviewer, BM	Records human override enforcing reason code taxonomy and appends audit log.
`GET /api/v1/governance/health`	Reviewer, BM	Returns rule override rates and diagnostic recommendations.
`POST /api/v1/governance/backtest`	Brand Manager	Simulates candidate rule proposal against historical draft corpus.
`GET /api/v1/transparency/disclosure`	Workspace Member	Returns AI provider telemetry, token counts, and retention policy notes.

7. Frontend Architecture & Neo-Brutalist Design System

Canon's user interface is styled under a strict, purposeful **Neo-Brutalist Design System**. It deliberately rejects generic AI dark-mode templates in favor of high-contrast, editorial clarity and tactile visual depth.

Token / Class	CSS Values & Specification	Visual Purpose
Canvas Surface	`background-color: #F4F0EA !important`	Warm editorial paper beige canvas providing a calm, authentic baseline.
`.neo-box`	`bg-white border-[2.5px] border-black shadow-[4px_4px_0px_#000000]`	Standard container card for bento items, queue listings, and rule panels.
`.neo-box-prominent`	`bg-white border-[3px] border-black shadow-[6px_6px_0px_#000000]`	Hero container card, modal dialogues, and primary callout sections.
`.neo-btn-teal`	`bg-[#00C2B2] border-2 border-black shadow-[3px_3px_0px_#000] font-bold`	Primary positive action button (e.g. Pre-flight Check, Adopt Policy).
`.neo-btn-yellow`	`bg-[#FFE500] border-2 border-black shadow-[3px_3px_0px_#000] font-bold`	Notice banners, highlighted version indicators, and guided tour steps.
`.neo-btn-magenta`	`bg-[#FF0055] border-2 border-black text-white shadow-[3px_3px_0px_#000]`	Critical override indicators, expired claims, and urgent warnings.

8. The 4-Step Evaluator Lifecycle Tour (/demo)

Canon includes a purpose-built interactive evaluator tour at **/demo** allowing stakeholders and judges to experience the entire pre-publication governance lifecycle in under 3 minutes:

Step 1: Inspect Policy Baseline & Claims	The evaluator explores Policy Baseline v1.0 (16 active rules) and inspects the Temporal Claims Ledger. They witness **CLAIM-001** ('#1 banking app with 99.99% uptime') flagged with an explicit EXPIRED status.
Step 2: Run Verification on Flawed Draft	The evaluator executes the pre-flight verification on a sample draft copy. The engine immediately flags 4 distinct violations: Banking Charter term, deceptive credit phrase, slang phrase, and the expired claim.
Step 3: Review Double Receipts & Override	The evaluator views the side-by-side Double Receipts (Receipt A: character span offsets; Receipt B: verbatim policy quote). They execute a human override selecting an authorized reason code (APPROVED_EXCEPTION).
Step 4: Backtest Simulation & Adopt Policy v1.1	The evaluator observes Rule TERM-007's 37.5% override rate, simulates candidate proposal PROP-01 across historical drafts (demonstrating false-positive reduction), and commits human adoption to Policy v1.1.

9. Key Source Files & Implementation Directory

For code reviews and repository navigation, here are the most critical implementation files in the repository:

`backend/app/main.py`	FastAPI gateway entrypoint, error handling middleware, CORS & route inclusions.

backend/app/services/verification_engine.py	Core 4-stage verification scanner generating Double Receipts A & B.
backend/app/services/backtest_service.py	Empirical backtesting simulation engine running proposals against draft corpus.
backend/app/api/v1/decisions.py	Reviewer queue triage with SQL pushdown filtering, pagination & override actions.
backend/app/api/v1/governance.py	Rule health snapshots, backtesting triggers, and human adoption to Policy v1.1.
backend/app/api/v1/transparency.py	AI provider telemetry and telemetry disclosures.
backend/scripts/seed_data.py	Comprehensive seed script configuring RBAC users, policy v1.0, claims, and drafts.
frontend/src/app/globals.css	Neo-Brutalist design tokens (#F4F0EA canvas, .neo-box, .neo-btn classes).
frontend/src/app/page.tsx	Homepage with framed prominent hero, edge proxy latency monitor, and bento cards.
frontend/src/app/demo/page.tsx	Interactive 4-step evaluator lifecycle tour with step scorecard.
frontend/src/app/review/page.tsx	Reviewer triage queue with side-by-side Double Receipts inspection.
frontend/src/app/policy/page.tsx	Active brand policy baseline and temporal claims registry viewer.
frontend/src/app/governance/page.tsx	Rule health diagnostic dashboard and empirical backtest simulator.
frontend/src/app/transparency/page.tsx	AI transparency and telemetry disclosure panel.